

期末大作业实验报告

多线程端口扫描编程

姓名：吕乐彬 学号：2023080904007

2026 年 5 月 18 日

课程名称：网络安全协议综合实验

实验名称：端口扫描编程（要求图形界面、多线程 2-3 线程，并测试 Radius 1812 和 1813 端口）

难度值：5

开发语言：C#

开发环境：Windows 11 + Visual Studio 2026 + .NET Framework + Windows Forms

目录

1	实验名称	4
2	实验内容和目的	4
2.1	实验内容	4
2.2	实验目的	4
3	实验原理	4
3.1	端口与网络服务	4
3.2	端口扫描基本思想	5
3.3	TCP connect 扫描原理	6
3.4	UDP 探测原理	7
3.5	多线程扫描原理	7
4	实验器材（设备、软件环境）	8
4.1	硬件环境	8
4.2	软件环境	9
5	实验设计思路	9
5.1	总体架构设计	9
5.2	图形界面设计	11
5.3	Radius 端口一键设置设计	11
5.4	扫描任务队列设计	12
5.5	工作线程设计	12
5.6	WorkerLoop 扫描流程设计	12
5.7	TCP 扫描模块设计	13
5.8	UDP 探测模块设计	14
5.9	界面更新与线程安全设计	14
5.10	RadiusMockServer 设计	14
6	实验步骤	15
6.1	打开项目并生成解决方案	15
6.2	启动 RadiusMockServer	16
6.3	启动 PortScannerGUI 并设置参数	16
6.4	执行扫描并记录结果	17
7	实验数据及结果分析	18
7.1	实验输入参数	18
7.2	扫描结果	18

7.3 结果分析	19
8 实验补充与拓展*	19
8.1 扩展后的程序结构	20
8.2 灵活端口表达式输入扩展	20
8.3 常见端口知识库扩展与扫描场景预设	21
8.4 扫描结果展示改进	22
8.5 并发扫描数量设置	22
8.6 扫描结果过滤功能	23
9 实验结论、心得体会和改进建议	23
9.1 实验结论	23
9.2 心得体会	24
9.3 改进建议	24
A 附录：主要源码（非拓展，实验要求部分）	25
A.1 PortScannerGUI/MainForm.cs	25
A.2 PortScannerGUI/Program.cs	41
A.3 RadiusMockServer/Program.cs	42

1 实验名称

多线程端口扫描编程与 Radius 1812/1813 端口测试。

2 实验内容和目的

2.1 实验内容

本实验完成一个基于 C# Windows Forms 的图形化多线程端口扫描程序。程序能够由用户输入目标主机、起始端口、结束端口、线程数、超时时间和扫描协议，并通过 2-3 个工作线程对目标端口进行并发扫描。实验重点测试 Radius 服务常用端口 1812 和 1813，验证扫描器能否正确识别 TCP 与 UDP 端口状态。

本实验包含两个可执行程序：

1. **PortScannerGUI**: 图形化端口扫描器，是本实验的核心程序。
2. **RadiusMockServer**: Radius 端口模拟服务，用于在本机监听 TCP/UDP 1812 和 1813 端口，方便验证扫描器功能。

2.2 实验目的

本实验的目的包括：

1. 理解 IP 地址、端口、TCP、UDP 和端口扫描之间的关系。
2. 掌握 TCP connect 端口扫描的基本原理。
3. 理解 UDP 扫描结果中 Open|Filtered 状态产生的原因。
4. 掌握 C# 中 TcpClient、UdpClient 和 Socket 网络编程的基本用法。
5. 掌握 Windows Forms 图形界面程序设计方法。
6. 掌握使用 Thread 和 ConcurrentQueue 实现多线程任务调度的方法。
7. 完成 Radius 1812 与 1813 端口测试，并对实验结果进行分析。

3 实验原理

3.1 端口与网络服务

在 TCP/IP 网络中，IP 地址用于定位网络中的一台主机，端口号用于定位该主机上的某个具体服务。可以理解为：

概念	作用
IP 地址	确定是哪一台主机
端口号	确定这台主机上的哪一个服务

端口号范围为 0 到 65535。常见服务端口如表 1 所示。

表 1: 常见端口及服务

端口号	协议	常见服务
80	TCP	HTTP Web 服务
443	TCP	HTTPS 安全 Web 服务
53	TCP/UDP	DNS 域名解析服务
1812	UDP/TCP	Radius 认证服务
1813	UDP/TCP	Radius 计费服务

Radius 全称为 Remote Authentication Dial-In User Service，常用于 VPN、无线网络、校园网、网络设备登录等场景中的认证、授权和计费。本实验关注的两个端口为：

- **1812**: Radius Authentication，主要用于用户认证。
- **1813**: Radius Accounting，主要用于计费和使用记录。

3.2 端口扫描基本思想

端口扫描的本质是：扫描器对目标主机的某个端口发起连接或探测请求，根据目标主机的响应情况判断端口状态。端口可能处于开放、关闭、超时或被过滤等状态。

表 2: 端口状态含义

状态	含义
Open	端口开放，目标主机上有服务正在监听该端口。
Closed	目标主机可达，但该端口没有服务监听。
Filtered/Timeout	扫描请求超时，可能被防火墙过滤，也可能是目标无响应。
Open Filtered	常见于 UDP 扫描，没有收到明确响应，因此无法完全区分开放还是被过滤。

端口扫描基本过程如图 1 所示。

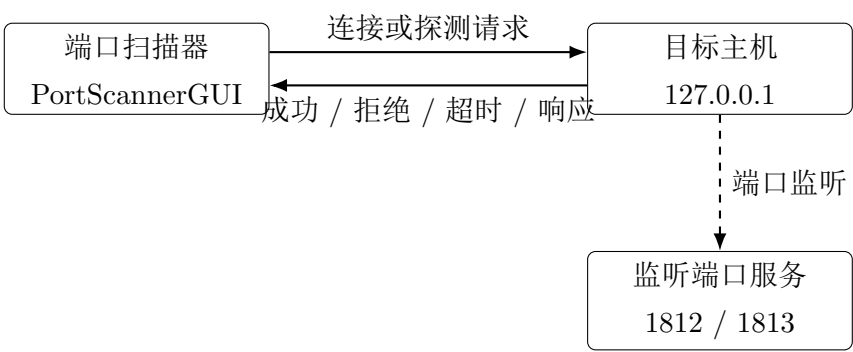


图 1: 端口扫描基本过程示意图

3.3 TCP connect 扫描原理

TCP 是面向连接的传输层协议。客户端访问服务器时，需要先建立连接。TCP connect 扫描正是利用这一特性，通过尝试连接目标端口来判断端口是否开放。

本实验中，TCP 扫描使用 C# 的 `TcpClient` 实现，其判断逻辑如下：

- 1. 调用 `BeginConnect(host, port, ...)` 尝试连接目标端口。
- 2. 使用 `AsyncWaitHandle.WaitOne(timeoutMs)` 等待连接结果。
- 3. 如果在超时时间内连接成功，则端口状态为 `Open`。
- 4. 如果连接失败或抛出异常，则端口可能为 `Closed` 或 `Closed/Error`。
- 5. 如果等待超时，则端口状态为 `Filtered/Timeout`。

TCP connect 扫描逻辑如图 2 所示。

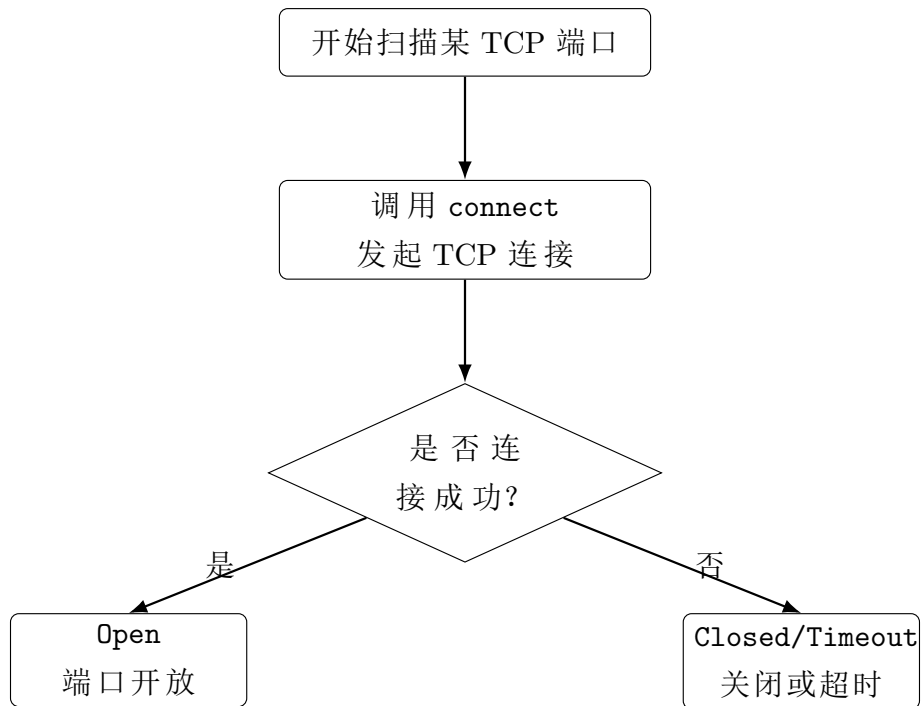


图 2: TCP connect 扫描原理

3.4 UDP 探测原理

UDP 是无连接协议，发送 UDP 数据前不需要建立连接。因此 UDP 扫描不能像 TCP connect 扫描那样通过“连接成功”直接判断端口开放。本实验中的 UDP 扫描采用“发送探测包并等待响应”的方式。

程序向目标 UDP 端口发送字符串：

RADIUS-PROBE

然后等待目标端口返回数据。判断逻辑如下：

1. 如果收到 UDP 响应，则判断端口为 Open。
2. 如果收到端口不可达或连接重置，则判断端口为 Closed。
3. 如果超时无响应，则判断为 Open|Filtered。

需要注意的是，真实 UDP 服务收到未知探测包后可能不会回复，所以 UDP 扫描结果需要谨慎分析。本实验为了稳定验证功能，使用 RadiusMockServer 对 UDP 1812 和 1813 主动返回 RADIUS-MOCK-OK 响应。

3.5 多线程扫描原理

端口扫描属于典型的 I/O 等待型任务。连接目标端口时，程序经常需要等待网络返回结果或等待超时。如果使用单线程逐个端口扫描，效率较低。多线程扫描的思想

是：将待扫描端口放入一个线程安全任务队列，再启动多个工作线程同时从队列中取任务并执行扫描。

本实验使用 `ConcurrentQueue<int>` 保存待扫描端口，用 2-3 个 `Thread` 对端口进行并发扫描。多线程结构如图 3 所示。

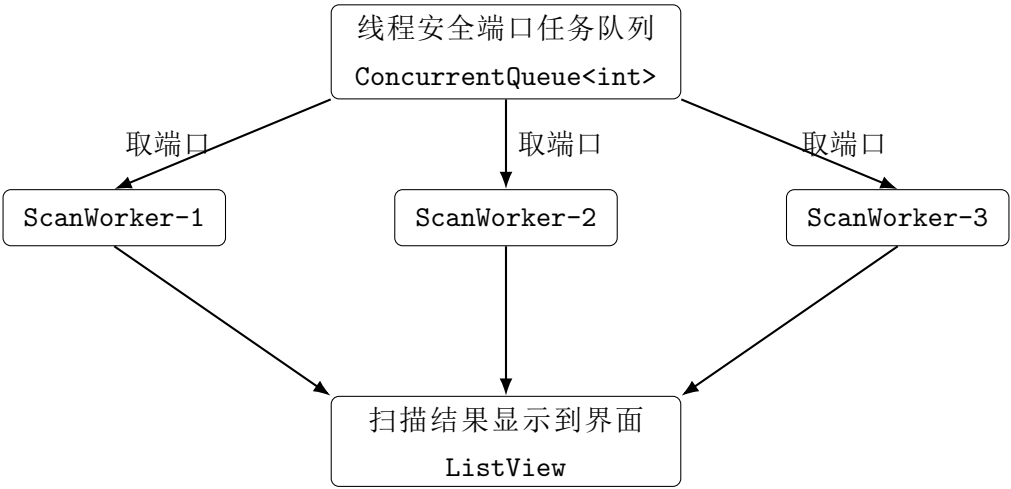


图 3: 多线程端口扫描结构

4 实验器材（设备、软件环境）

4.1 硬件环境

本实验在 Windows 11 主机上完成，硬件环境如表 3 所示。

表 3: 实验硬件环境

项目	配置
操作系统	Windows 11
处理器	AMD Ryzen 7 7840H w/ Radeon 780M Graphics
网络环境	本机回环地址 127.0.0.1，用于本地端口测试

4.2 软件环境

表 4: 实验软件环境

项目	说明
开发工具	Visual Studio 2026
开发语言	C#
图形界面框架	Windows Forms
运行框架	.NET Framework
主要命名空间	System.Net、System.Net.Sockets、 System.Threading、System.Windows.Forms

5 实验设计思路

本节是本实验的重点。整个程序采用“辅助服务端 + 图形化扫描客户端”的结构：先用 RadiusMockServer 在本机模拟 Radius 端口服务，再用 PortScannerGUI 对这些端口进行扫描。这样做的好处是实验环境可控，端口开放状态明确，便于验证扫描器是否正确。

5.1 总体架构设计

程序总体架构如图 4 所示。

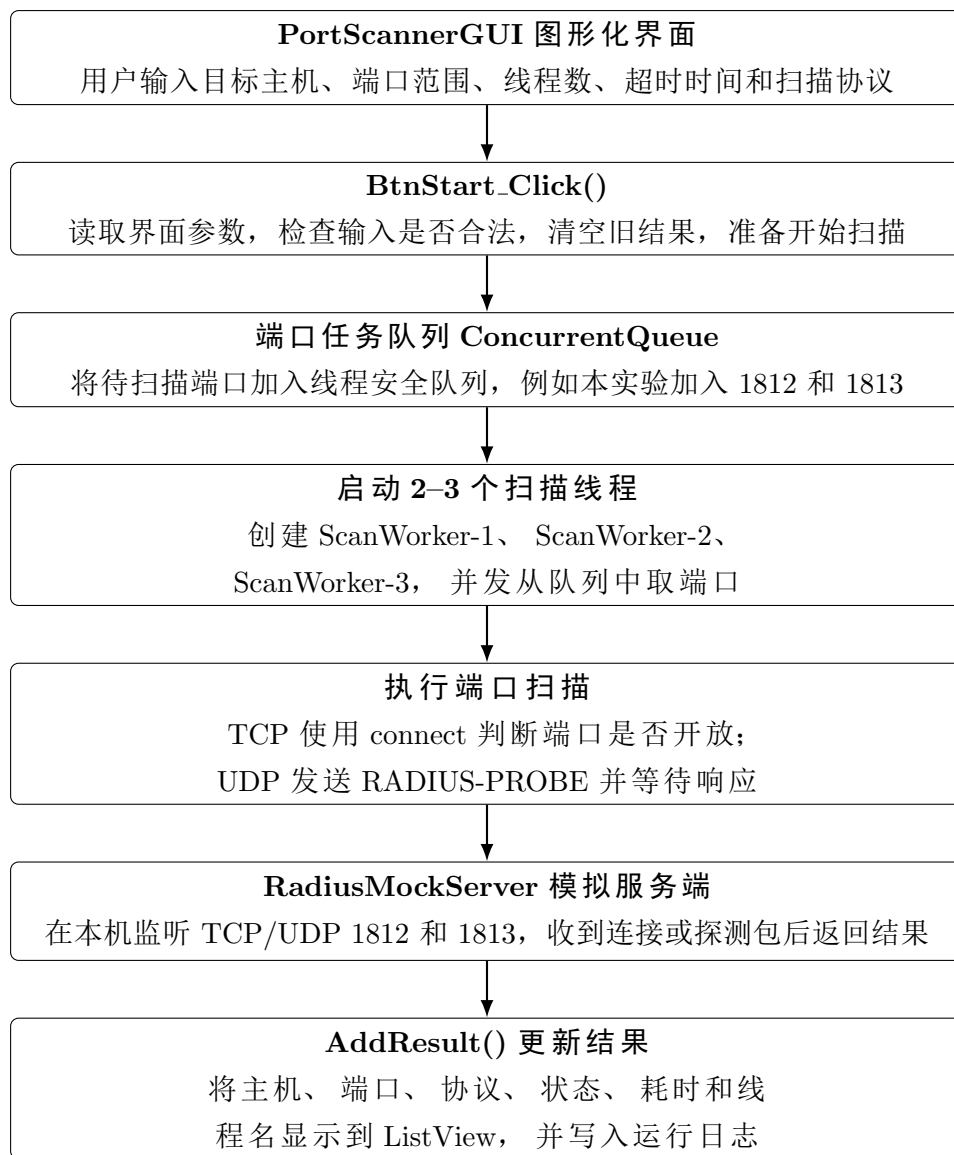


图 4: 程序总体架构设计

整个流程可以概括为:

1. 用户在图形界面输入目标主机、端口范围、线程数、超时时间和协议类型。
2. 点击“开始扫描”后，程序将端口范围中的每个端口加入线程安全队列。
3. 程序创建 2-3 个工作线程，每个线程循环从队列中取出端口。
4. 工作线程根据协议选项执行 TCP 扫描、UDP 扫描或 TCP+UDP 扫描。
5. 每次扫描完成后，程序将结果封装为 `ScanResult` 对象。
6. UI 线程将扫描结果添加到 `ListView` 中，并更新进度条和日志。

5.2 图形界面设计

图形界面由 MainForm 类实现。程序没有使用拖拽式界面设计器，而是在 BuildUi() 方法中通过代码创建控件。主要控件如表 5 所示。

表 5: 主要界面控件设计

控件变量	控件类型	功能
txtHost	TextBox	输入目标主机或 IP 地址，默认值为 127.0.0.1。
nudStartPort	NumericUpDown	输入起始端口，默认设置为 1812。
nudEndPort	NumericUpDown	输入结束端口，默认设置为 1813。
nudThreadCount	NumericUpDown	输入扫描线程数，最小值为 2，最大值为 3，满足题目要求。
cboProtocol	ComboBox	选择 TCP、UDP 或 TCP+UDP。
btnStart	Button	开始扫描。
lvResults	ListView	显示端口、协议、状态、耗时和线程名等扫描结果。
txtLog	TextBox	显示运行日志。

其中，线程数控件被限制为：

```
1 nudThreadCount.Minimum = 2;
2 nudThreadCount.Maximum = 3;
3 nudThreadCount.Value = 3;
```

这样可以从界面层面保证程序符合“多线程 2-3 线程”的实验要求。

5.3 Radius 端口一键设置设计

为了贴合题目要求，界面中加入“Radius 1812/1813”按钮。用户点击后，程序自动设置实验所需参数：

```
1 nudStartPort.Value = 1812;
2 nudEndPort.Value = 1813;
3 nudThreadCount.Value = 3;
4 cboProtocol.SelectedIndex = 2; // TCP+UDP
```

该按钮的设计意义是减少手动输入错误，保证实验测试目标明确，即扫描 Radius 1812 和 1813 端口。

5.4 扫描任务队列设计

当用户点击“开始扫描”后，程序会读取起始端口和结束端口，并将端口逐个加入 `ConcurrentQueue<int>` 队列。例如本实验端口范围为 1812–1813，则队列内容为：

1812, 1813

使用 `ConcurrentQueue<int>` 的原因是多个线程会同时访问该队列。普通队列在多线程环境下可能出现数据竞争，而 `ConcurrentQueue` 是线程安全集合，可以保证多个线程同时取任务时不会破坏队列状态。

核心逻辑如下：

```
1 portQueue = new ConcurrentQueue<int>();
2 for (int port = startPort; port <= endPort; port++)
3 {
4     portQueue.Enqueue(port);
5 }
```

5.5 工作线程设计

程序根据用户设置的线程数启动多个扫描线程。每个线程的名字为：

ScanWorker-1、ScanWorker-2、ScanWorker-3

线程启动逻辑如下：

```
1 for (int i = 1; i <= workerCount; i++)
2 {
3     string workerName = "ScanWorker-" + i;
4     Thread worker = new Thread(() => WorkerLoop(workerName, scanCts.Token));
5     worker.IsBackground = true;
6     worker.Name = workerName;
7     worker.Start();
8 }
```

每个线程都会进入 `WorkerLoop()` 方法。在该方法中，线程不断从队列中取出端口。如果队列为空，线程结束；如果用户点击“停止”，线程也会根据取消信号退出。

5.6 WorkerLoop 扫描流程设计

`WorkerLoop()` 是扫描线程真正执行任务的地方。其流程如图 5 所示。

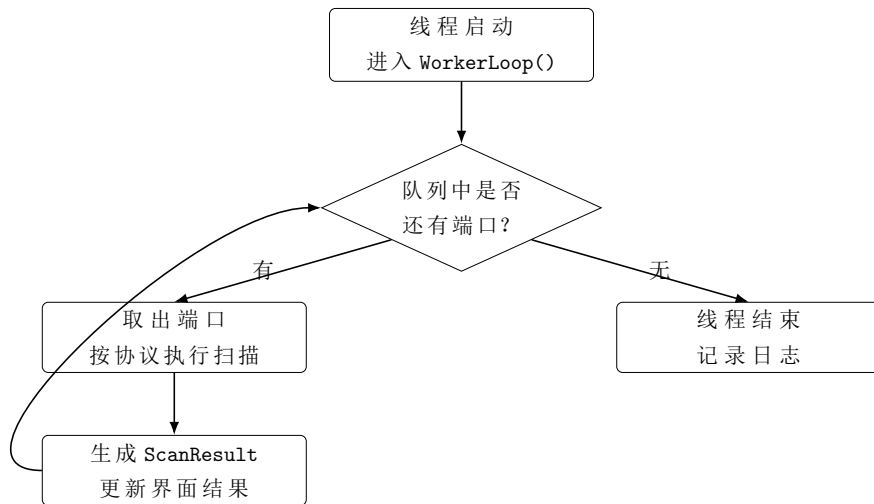


图 5: WorkerLoop() 工作线程流程

在 WorkerLoop() 中, 如果用户选择 TCP, 则调用 ScanTcp(); 如果选择 UDP, 则调用 ScanUdp(); 如果选择 TCP+UDP, 则同一个端口会分别执行 TCP 和 UDP 两种扫描。因此本实验扫描两个端口时, 最终显示 4 条结果:

1812/TCP, 1812/UDP, 1813/TCP, 1813/UDP

5.7 TCP 扫描模块设计

TCP 扫描方法为 ScanTcp()。其核心思想是使用 TcpClient 发起连接, 并通过等待连接结果判断端口状态。

关键代码逻辑为:

```

1 using (TcpClient client = new TcpClient())
2 {
3     IAsyncResult ar = client.BeginConnect(host, port, null, null);
4     bool connected = ar.AsyncWaitHandle.WaitOne(timeoutMs);
5
6     if (!connected)
7     {
8         client.Close();
9         return FilteredOrTimeout;
10    }
11
12    client.EndConnect(ar);
13    return Open;
14 }
  
```

设计上使用 `BeginConnect()` 而不是直接同步连接，是为了能够设置超时时间。如果直接同步连接，当目标端口无响应时，界面可能长时间等待，影响用户体验。

5.8 UDP 探测模块设计

UDP 扫描方法为 `ScanUdp()`。程序使用 `UdpClient` 连接目标 UDP 端口，发送字符串 `RADIUS-PROBE`，然后等待响应：

```
1 byte[] probe = Encoding.ASCII.GetBytes("RADIUS-PROBE");
2 udp.Send(probe, probe.Length);
3 byte[] reply = udp.Receive(ref remote);
```

如果收到响应，说明目标 UDP 端口有服务回复，判断为 `Open`。如果等待超时，则返回 `Open|Filtered`。本实验中，`RadiusMockServer` 会对 UDP 1812 和 1813 返回 `RADIUS-MOCK-OK`，因此实验结果中 UDP 端口均显示为 `Open`。

5.9 界面更新与线程安全设计

Windows Forms 程序中，子线程不能直接随意修改 UI 控件，否则可能产生跨线程访问异常。因此程序封装了 `Ui(Action action)` 方法，用于将界面更新操作切换到 UI 线程执行。

核心思想如下：

```
1 private void Ui(Action action)
2 {
3     if (InvokeRequired)
4         BeginInvoke(action);
5     else
6         action();
7 }
```

扫描线程完成任务后，不直接操作 `ListView`，而是通过 `AddResult()` 调用 UI 更新逻辑。这样既能保证多线程扫描效率，又能保证界面更新安全。

5.10 RadiusMockServer 设计

`RadiusMockServer` 的作用是为实验提供可控的测试端口。程序启动后创建 4 个监听线程：

表 6: RadiusMockServer 监听线程

线程名称	协议/端口	功能
TCP-1812	TCP 1812	接收 TCP 连接并返回测试字符串。
TCP-1813	TCP 1813	接收 TCP 连接并返回测试字符串。
UDP-1812	UDP 1812	接收 UDP 探测并返回 RADIUS-MOCK-OK 1812。
UDP-1813	UDP 1813	接收 UDP 探测并返回 RADIUS-MOCK-OK 1813。

该程序不是完整的 Radius 协议实现，而是实验辅助程序。其目的在于让 1812 和 1813 端口处于可检测的开放状态，从而验证扫描器的端口判断逻辑。

6 实验步骤

6.1 打开项目并生成解决方案

1. 使用 Visual Studio 2026 打开 PortScannerLab.sln。
2. 确认解决方案中包含 PortScannerGUI 与 RadiusMockServer 两个项目。
3. 点击“生成”菜单中的“生成解决方案”。
4. 确认项目生成成功，没有编译错误。

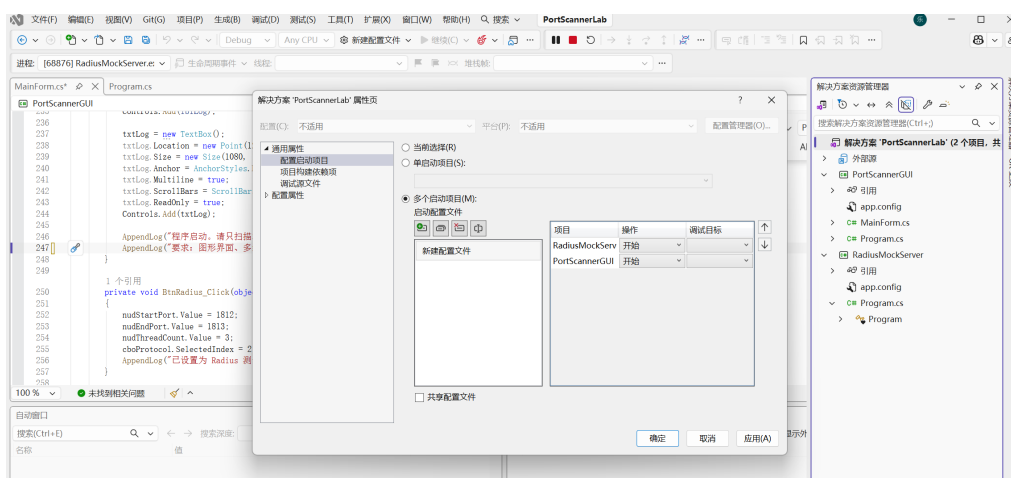


图 6: Visual Studio 打开项目并配置

6.2 启动 RadiusMockServer

1. 右键 RadiusMockServer 项目，选择“设为启动项目”。
2. 按 Ctrl + F5 运行程序。
3. 观察控制台窗口，确认 TCP/UDP 1812 和 1813 均处于监听状态。



```
Radius 1812/1813 测试服务启动。
作用：在本机打开 TCP/UDP 1812、1813，便于端口扫描程序截图验证。
说明：这是实验辅助程序，不实现真正的 Radius 协议认证。
按 Enter 键停止。
已启动监听线程：TCP-1812
已启动监听线程：TCP-1813
已启动监听线程：UDP-1812
已启动监听线程：UDP-1813
UDP 1813 正在监听。
UDP 1812 正在监听。
TCP 1813 正在监听。
TCP 1812 正在监听。
```

图 7: RadiusMockServer 监听状态

6.3 启动 PortScannerGUI 并设置参数

1. 右键 PortScannerGUI 项目，选择“设为启动项目”。
2. 按 Ctrl + F5 运行图形化扫描器。
3. 点击“Radius 1812/1813”按钮，自动设置扫描参数。
4. 确认目标主机为 127.0.0.1，起始端口为 1812，结束端口为 1813，线程数为 3，协议为 TCP+UDP。

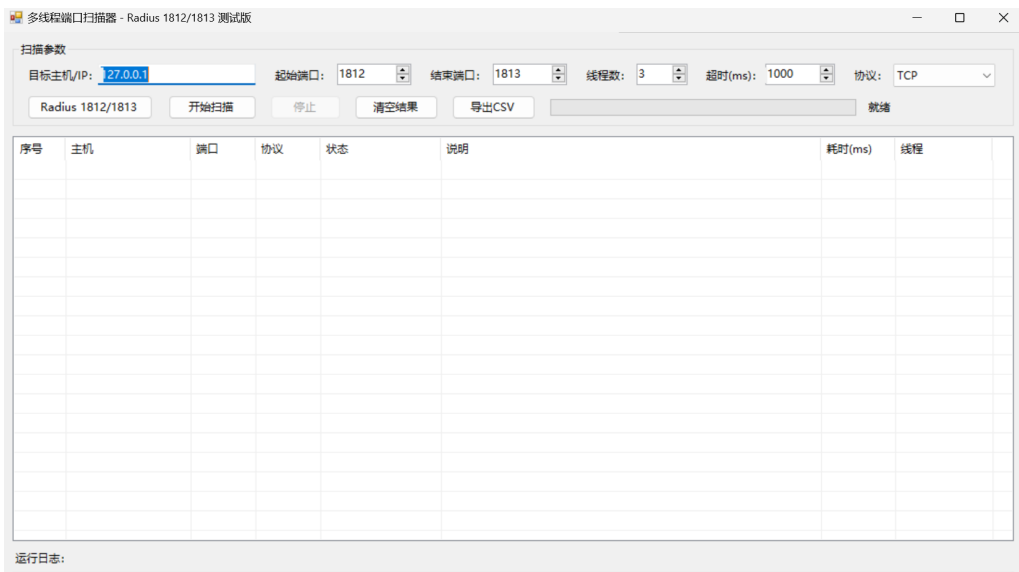


图 8: PortScannerGUI 界面

6.4 执行扫描并记录结果

- 1. 点击“开始扫描”按钮。
- 2. 等待进度条达到 100%。
- 3. 查看结果列表中 1812 和 1813 端口的 TCP/UDP 状态。
- 4. 截图保存扫描结果。
- 5. 可选：点击“导出 CSV”按钮，将扫描结果保存为 CSV 文件。

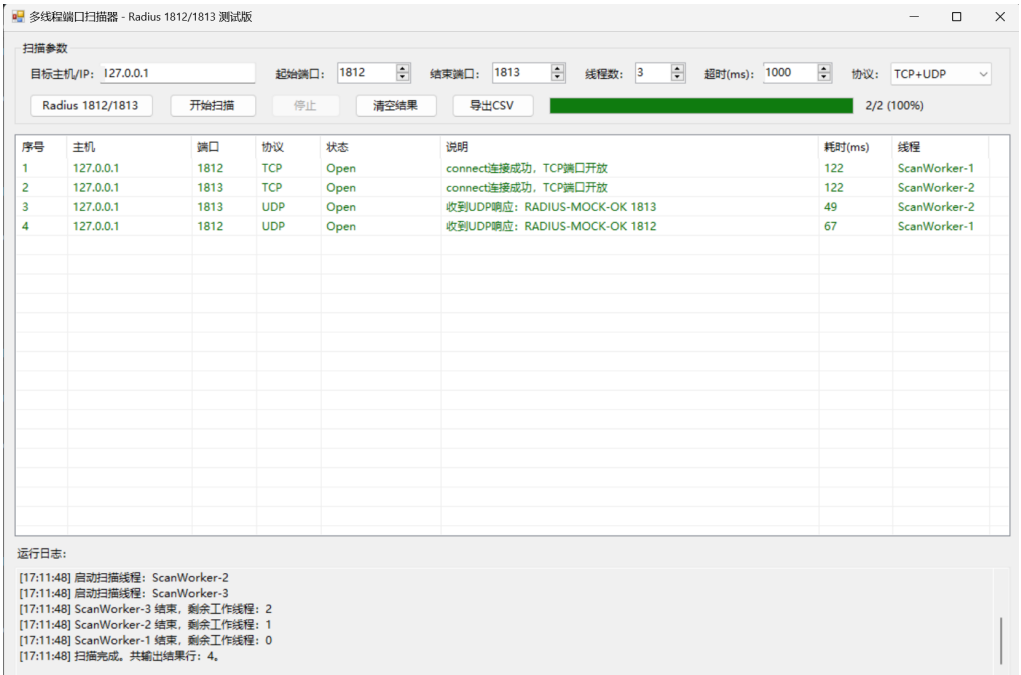


图 9: 扫描结果显示

7 实验数据及结果分析

7.1 实验输入参数

本次实验设置的扫描参数如表 7 所示。

表 7: 扫描输入参数

参数	设置值
目标主机/IP	127.0.0.1
起始端口	1812
结束端口	1813
线程数	3
超时时间	1000 ms
扫描协议	TCP+UDP

7.2 扫描结果

实验扫描结果如表 8 所示。

表 8: Radius 1812/1813 扫描结果

序号	主机	端口	协议	状态	说明
1	127.0.0.1	1812	TCP	Open	connect 连接成功, TCP 端口开放
2	127.0.0.1	1813	TCP	Open	connect 连接成功, TCP 端口开放
3	127.0.0.1	1812	UDP	Open	收到 UDP 响应: RADIUS-MOCK-OK 1812
4	127.0.0.1	1813	UDP	Open	收到 UDP 响应: RADIUS-MOCK-OK 1813

7.3 结果分析

从 TCP 扫描结果看, 1812 和 1813 端口均显示为 `Open`, 说明 `RadiusMockServer` 的 TCP 监听线程已经成功启动。扫描器使用 TCP connect 方法对目标端口发起连接, 连接成功后判断端口开放, 因此 TCP 1812 和 TCP 1813 的结果正确。

从 UDP 扫描结果看, 1812 和 1813 端口也均显示为 `Open`。说明栏显示扫描器收到了 `RADIUS-MOCK-OK 1812` 与 `RADIUS-MOCK-OK 1813`, 说明 UDP 探测数据成功到达模拟服务端, 并且服务端成功返回响应。因此 UDP 1812 和 UDP 1813 的结果也正确。

本次实验中, 端口范围为 1812–1813, 共 2 个端口。由于协议选择为 TCP+UDP, 所以每个端口会分别进行 TCP 扫描和 UDP 探测, 最终得到 4 条结果。界面进度条显示 2/2, 是因为进度按端口数量统计, 而不是按协议探测次数统计, 因此该显示是合理的。

从多线程角度看, 程序启动了 3 个扫描线程。由于本次只扫描两个端口, 实际拿到端口任务的主要是其中两个线程, 另一个线程可能启动后发现任务队列已空而结束。这并不影响多线程设计的正确性。如果将扫描范围扩大到 1811–1813 或更大范围, 可以更明显地观察到三个线程共同参与扫描。

8 实验补充与拓展*

在完成课程实验要求的基础上, 我对本端口扫描程序进行了进一步扩展。

将该项目扩展为一个面向 Windows 桌面环境的轻量级端口扫描工具。程序在保留原实验要求的图形界面、多线程扫描、TCP/UDP 探测以及 Radius 1812/1813 端口测试功能的基础上, 进一步增加了端口知识库、扫描场景预设、结果过滤、CSV 导出、Markdown 摘要复制等功能, 使实验程序具有更好的实用性和可展示性。

8.1 扩展后的程序结构

扩展后的项目结构更加清晰，主要分为图形界面、扫描核心、结果模型、服务知识库、导出模块和 Radius 模拟服务几个部分。其结构如下：

从结构上看，扩展后的程序将界面逻辑和扫描核心逻辑进行了分离。这样做的好处是后续可以在不大幅修改扫描核心的情况下，继续增加命令行版本、单元测试、JSON 导出、资产基线对比等功能。

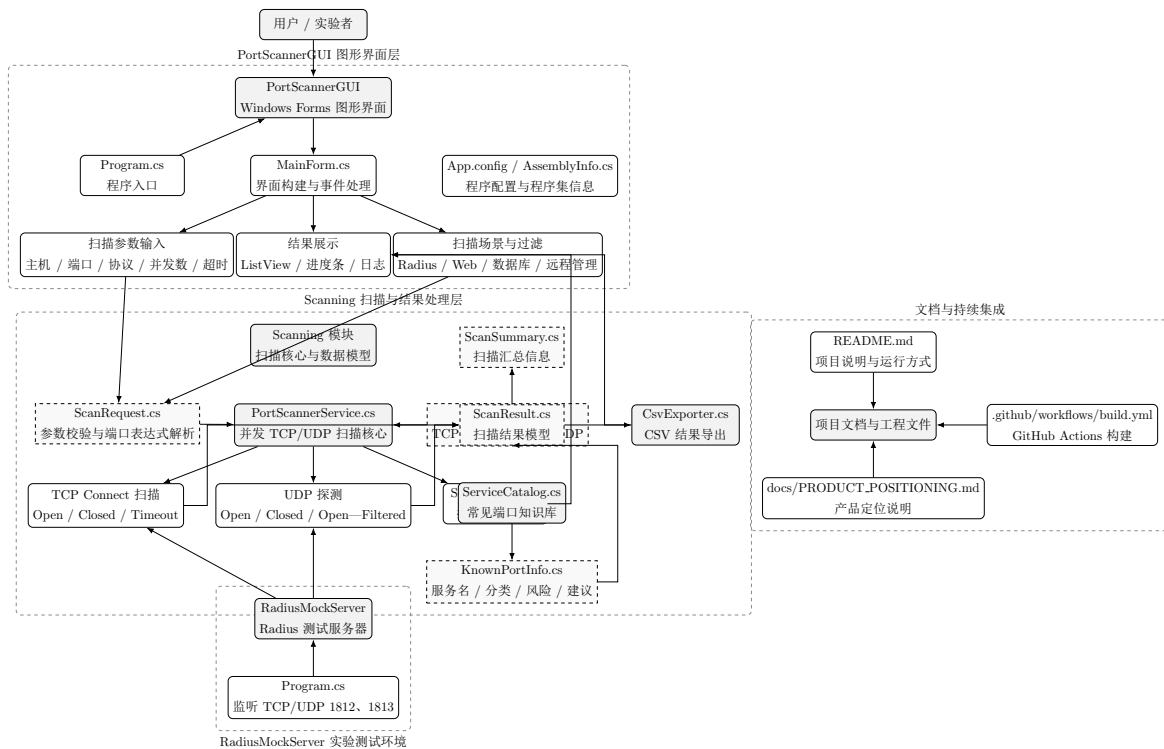


图 10: PortScannerLab 的程序结构图

8.2 灵活端口表达式输入扩展

在原始实验设计中，端口扫描通常采用“起始端口”和“结束端口”的方式进行输入，这种方式适合连续端口范围扫描，但在实际使用中不够灵活。例如，用户有时只希望扫描 Radius 实验端口 1812-1813，有时又希望同时检查多个远程管理相关端口，如 22,23,135,139,445,3389,5900,5985,5986。如果仍然采用单一起止端口的输入方式，就会扫描大量无关端口，影响实验效率。

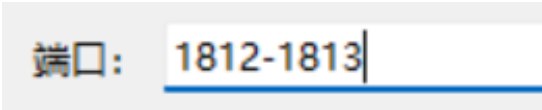
因此，在扩展版本中，我将端口输入方式改为端口表达式输入。用户既可以输入连续端口范围，也可以输入多个离散端口，甚至可以将两种方式组合使用。例如：

1812-1813

表示扫描 1812 到 1813 之间的连续端口；而：

22,23,135,139,445,3389,5900,5985,5986

则表示扫描多个不连续的指定端口。



(a) 连续端口范围输入示例：1812-1813



(b) 离散端口集合输入示例

图 11: 扩展后的端口表达式输入方式

这种设计使程序比原实验中的端口范围输入更加灵活。一方面，它可以继续满足 Radius 实验中对 1812 和 1813 端口的连续范围扫描；另一方面，也可以用于实际场景中的重点端口检查，例如远程管理端口、Web 服务端口、数据库端口等。这样既减少了无意义端口扫描，又提升了程序在真实排查场景中的实用性。

8.3 常见端口知识库扩展与扫描场景预设

为了减少用户手动输入端口的工作量，扩展版本中加入了扫描场景预设。用户可以在界面中直接选择课程/Radius 实验、运维常用暴露面、Web/开发服务、数据库与缓存、远程管理等场景，程序会自动填入对应的常见端口集合。

在代码实现上，场景下拉框由 MainForm.cs 中的 cboProfile 控件实现，不同场景会对应不同的端口表达式。例如选择“远程管理”后，程序会自动填入 22,23,135,139,445,3389, 这些端口覆盖了 SSH、Telnet、SMB、RDP、VNC、WinRM 等常见远程管理服务。

同时，程序还在 ServiceCatalog.cs 中维护了常见端口知识库，用于记录端口号、协议、服务名称、风险等级和安全建议。扫描完成后，程序会根据端口号和协议匹配对应的服务信息，使扫描结果更容易理解。

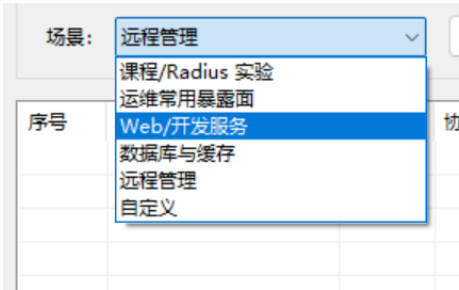


图 12: 扫描场景预设

8.4 扫描结果展示改进

扩展版本对扫描结果表格进行了改进。原始端口扫描程序通常只显示端口是否开放，而本程序在结果中增加了服务名称、风险等级、说明和安全建议等字段。这样用户不仅可以知道某个端口的状态，还可以进一步理解该端口可能对应的服务以及是否存在安全风险。

在代码实现上，每条扫描结果由 `ScanResult.cs` 表示，其中包含主机、端口、协议、状态、服务名称、风险等级、说明、建议、耗时和工作器等信息。扫描过程中，`PortScannerService.cs` 会完成 TCP 或 UDP 探测，并结合 `ServiceCatalog.cs` 生成更完整的结果信息。最后，`MainForm.cs` 将这些字段显示到结果表格中。

例如，在远程管理场景下，程序可以识别出 135 端口为 MS RPC、445 端口为 SMB、3389 端口为 RDP，并给出 High 或 Critical 等风险提示。相比只显示 Open 或 Closed，这种展示方式更适合后续分析和写入实验报告。

扫描参数									
目标主机/IP: 127.0.0.1		端口: 22,23,135,139,445,3389,5900,5985,5986		并发数: 32	超时(ms): 1000	协议: TCP			
场景: 远程管理	应用	Radius	开始扫描	停止	清空结果	导出CSV	复制摘要	全部结果	9/9 (100%)
序号	主机	端口	协议	状态	服务	风险	说明	建议	
1	127.0.0.1	135	TCP	Open	MS RPC	High	connect 连接成功, TCP 端口开放	Windows RPC 不应暴露在互联网。	
2	127.0.0.1	445	TCP	Open	SMB	Critical	connect 连接成功, TCP 端口开放	SMB 不应暴露在互联网, 建议限制来源并启用。	
3	127.0.0.1	3389	TCP	Open	RDP	High	connect 连接成功, TCP 端口开放	限制来源地址, 启用 NLA/MFA, 并监控暴力破解。	
4	127.0.0.1	5985	TCP	Filtered/Timeout	Unknown	None	TCP connect 超时, 端口可能被过滤或主机无响应	无需处理; 端口未显示为开放。	
5	127.0.0.1	23	TCP	Filtered/Timeout	Telnet	None	TCP connect 超时, 端口可能被过滤或主机无响应	无需处理; 端口未显示为开放。	
6	127.0.0.1	5900	TCP	Filtered/Timeout	VNC	None	TCP connect 超时, 端口可能被过滤或主机无响应	无需处理; 端口未显示为开放。	
7	127.0.0.1	5986	TCP	Filtered/Timeout	Unknown	None	TCP connect 超时, 端口可能被过滤或主机无响应	无需处理; 端口未显示为开放。	
8	127.0.0.1	22	TCP	Filtered/Timeout	SSH	None	TCP connect 超时, 端口可能被过滤或主机无响应	无需处理; 端口未显示为开放。	
9	127.0.0.1	139	TCP	Filtered/Timeout	NetBIOS	None	TCP connect 超时, 端口可能被过滤或主机无响应	无需处理; 端口未显示为开放。	

图 13: 改进后的扫描结果展示

8.5 并发扫描数量设置

扩展版本中加入了并发扫描数量设置，用户可以在界面中手动调整扫描并发数。该参数用于控制同时执行端口探测任务的工作线程数量，从而提高多端口扫描时的效率。

在代码实现上，`ScanRequest.cs` 中定义并发数范围为 1-256，界面中的并发数控件也使用该范围作为限制。因此，程序允许用户根据实验环境和目标端口数量灵活调整并发数。图中设置的并发数为 32，这是一个较为折中的默认值，既能提高扫描速度，又不会对本机或目标主机造成过大压力。

同时，程序在实际扫描时会根据端口数量自动限制工作线程数量，即实际并发数不会超过待扫描端口总数。这样可以避免在端口数量较少时创建过多无意义的扫描任务。



图 14: 并发扫描数量设置

8.6 扫描结果过滤功能

为了便于在大量扫描结果中快速定位重点端口，扩展版本中加入了结果过滤功能。用户可以通过界面中的下拉框选择不同的显示范围，包括全部结果、开放/不确定端口、Critical/High 高风险端口、仅开放端口和仅关闭端口。

在代码实现上，该功能由 `MainForm.cs` 中的 `cboResultFilter` 控件实现。当用户切换过滤条件时，程序会重新刷新结果列表，并通过 `IsResultVisible()` 方法判断每一条扫描结果是否应该显示。例如，“Critical/High” 选项只显示高风险端口，“仅开放” 选项只显示状态为 `Open` 的端口。

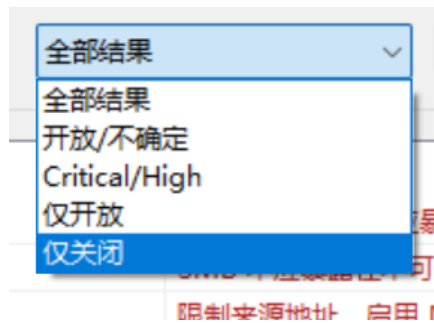


图 15: 扫描结果过滤功能

该功能可以减少无关结果对分析过程的干扰，使用户能够更快关注开放端口和高风险端口，提高实验结果分析效率。

9 实验结论、心得体会和改进建议

9.1 实验结论

本次实验完成了一个基于 C# Windows Forms 的图形化端口扫描工具。程序能够对指定主机和端口进行 TCP/UDP 扫描，并通过多线程方式提高扫描效率。在基础实验要求中，程序可以对 Radius 常用端口 1812 和 1813 进行检测，并配合 `RadiusMockServer` 完成本机实验环境验证。

在实验扩展部分，程序不再局限于简单的端口开放检测，而是进一步加入了端口表达式输入、扫描场景预设、常见端口知识库、风险等级提示、结果过滤、CSV 导出和 Markdown 摘要复制等功能。通过这些扩展，程序能够根据不同应用场景快速填入常见端口，并在扫描结果中显示服务名称、风险等级和安全建议，使扫描结果更加直观、易理解，也更适合写入实验报告。

实验结果表明，该程序能够完成基本端口扫描任务，并且能够较好地区分 `Open`、`Closed`、`Filtered/Timeout` 和 `Open|Filtered` 等状态。相比原始的端口扫描实验，扩展后的程序在可用性、可解释性和结果整理方面都有明显提升。

9.2 心得体会

通过本次实验，我对端口扫描的基本原理有了更深入的理解。TCP 扫描可以通过建立连接来判断端口是否开放，而 UDP 扫描由于协议本身不保证响应，因此扫描结果具有一定不确定性。例如，当 UDP 端口没有返回数据时，并不能简单判断端口一定关闭，因此程序中使用 `Open|Filtered` 表示开放或被过滤的可能状态。这个过程让我认识到，不同协议的扫描方式和结果解释存在明显差异。

在程序设计过程中，我也体会到图形化工具的重点不仅是“能扫描”，还要让用户“看得懂”。因此，我在扩展中加入了常见端口知识库和风险提示功能，使扫描结果不只是显示端口状态，还能说明该端口可能对应的服务及其安全影响。例如，445 端口对应 SMB 服务，3389 端口对应远程桌面服务，这些端口如果暴露在不可信网络中就需要重点关注。

另外，通过实现端口表达式解析、扫描场景预设和结果过滤功能，我认识到一个实验程序也可以向实用工具方向改进。相比命令行工具，图形化界面更适合课堂演示和实验报告截图；相比只输出原始结果的扫描器，本程序更强调结果解释和报告整理，这也是本次扩展中比较有特色的地方。

9.3 改进建议

虽然本程序已经完成了基础扫描和部分扩展功能，但仍然存在一些可以继续改进的地方。

首先，目前程序主要支持单个目标主机扫描，后续可以增加多主机或网段扫描功能。例如支持输入多个 IP 地址，或者支持简单的局域网网段扫描，使程序能够用于更完整的网络资产检查。

其次，当前的服务识别主要依赖端口号和少量 TCP Banner 信息，准确性仍然有限。后续可以继续完善服务识别逻辑，例如针对 HTTP、SSH、FTP、SMTP 等常见协议设计更细致的低扰动探测方式，从而提高服务判断的准确性。

第三，当前扫描结果已经支持 CSV 导出和 Markdown 摘要复制，但还可以进一步扩展为 HTML 报告或 LaTeX 表格导出。这样可以更方便地将扫描结果直接放入实验报告中，提高报告整理效率。

第四，界面还可以继续优化。例如可以增加更明显的风险统计区域，显示开放端口数量、高风险端口数量、不确定 UDP 端口数量等信息，使用户在扫描完成后能够快速了解整体风险情况。

最后，程序后续可以加入更严格的使用提示和扫描范围确认机制。端口扫描在真实网络中可能触发安全告警，因此工具应继续强调仅在本人主机、实验环境或已获得授权的网络中使用，避免不当扫描行为。

A 附录：主要源码（非拓展，实验要求部分）

A.1 PortScannerGUI/MainForm.cs

```
using System;
using System.Collections.Concurrent;
using System.Diagnostics;
using System.Drawing;
using System.IO;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;
using System.Windows.Forms;

namespace PortScannerGUI
{
    public enum ScanMode
    {
        TCP,
        UDP,
        TCPAndUDP
    }

    public class ScanResult
    {
        public string Host { get; set; }
        public int Port { get; set; }
        public string Protocol { get; set; }
        public string Status { get; set; }
        public string Message { get; set; }
        public long ElapsedMs { get; set; }
        public string Worker { get; set; }
    }

    public class MainForm : Form
    {
        private TextBox txtHost;
        private NumericUpDown nudStartPort;
        private NumericUpDown nudEndPort;
```

```
private NumericUpDown nudThreadCount;
private NumericUpDown nudTimeout;
private ComboBox cboProtocol;
private Button btnStart;
private Button btnStop;
private Button btnRadius;
private Button btnClear;
private Button btnExport;
private ProgressBar progressBar;
private Label lblProgress;
private ListView lvResults;
private TextBox txtLog;

private ConcurrentQueue<int> portQueue;
private CancellationTokenSource scanCts;
private int totalPorts;
private int completedPorts;
private int remainingWorkers;
private int resultIndex;
private string currentHost;
private int currentTimeoutMs;
private ScanMode currentMode;

public MainForm()
{
    Text = "多线程端口扫描器-LLB2023080904007-测试版";
    StartPosition = FormStartPosition.CenterScreen;
    Size = new Size(1120, 720);
    MinimumSize = new Size(1000, 620);

    BuildUi();
}

private void BuildUi()
{
    Font = new Font("Microsoft YaHei UI", 9F, FontStyle.Regular,
        ↪ GraphicsUnit.Point);

    GroupBox groupInput = new GroupBox();
    groupInput.Text = "扫描参数";
```

```
groupInput.Location = new Point(12, 10);
groupInput.Size = new Size(1080, 92);
groupInput.Anchor = AnchorStyles.Top | AnchorStyles.Left |
    ↪ AnchorStyles.Right;
Controls.Add(groupInput);

Label lblHost = new Label();
lblHost.Text = "目标主机/IP: ";
lblHost.Location = new Point(14, 28);
lblHost.AutoSize = true;
groupInput.Controls.Add(lblHost);

txtHost = new TextBox();
txtHost.Location = new Point(92, 24);
txtHost.Size = new Size(170, 24);
txtHost.Text = "127.0.0.1";
groupInput.Controls.Add(txtHost);

Label lblStart = new Label();
lblStart.Text = "起始端口: ";
lblStart.Location = new Point(280, 28);
lblStart.AutoSize = true;
groupInput.Controls.Add(lblStart);

nudStartPort = new NumericUpDown();
nudStartPort.Location = new Point(350, 24);
nudStartPort.Minimum = 0;
nudStartPort.Maximum = 65535;
nudStartPort.Value = 1812;
nudStartPort.Size = new Size(80, 24);
groupInput.Controls.Add(nudStartPort);

Label lblEnd = new Label();
lblEnd.Text = "结束端口: ";
lblEnd.Location = new Point(448, 28);
lblEnd.AutoSize = true;
groupInput.Controls.Add(lblEnd);

nudEndPort = new NumericUpDown();
nudEndPort.Location = new Point(518, 24);
```

```
nudEndPoint.Minimum = 0;
nudEndPoint.Maximum = 65535;
nudEndPoint.Value = 1813;
nudEndPoint.Size = new Size(80, 24);
groupInput.Controls.Add(nudEndPoint);

Label lblThread = new Label();
lblThread.Text = "线程数: ";
lblThread.Location = new Point(616, 28);
lblThread.AutoSize = true;
groupInput.Controls.Add(lblThread);

nudThreadCount = new NumericUpDown();
nudThreadCount.Location = new Point(673, 24);
nudThreadCount.Minimum = 2;
nudThreadCount.Maximum = 3;
nudThreadCount.Value = 3;
nudThreadCount.Size = new Size(55, 24);
groupInput.Controls.Add(nudThreadCount);

Label lblTimeout = new Label();
lblTimeout.Text = "超时(ms): ";
lblTimeout.Location = new Point(745, 28);
lblTimeout.AutoSize = true;
groupInput.Controls.Add(lblTimeout);

nudTimeout = new NumericUpDown();
nudTimeout.Location = new Point(812, 24);
nudTimeout.Minimum = 100;
nudTimeout.Maximum = 10000;
nudTimeout.Increment = 100;
nudTimeout.Value = 1000;
nudTimeout.Size = new Size(75, 24);
groupInput.Controls.Add(nudTimeout);

Label lblProtocol = new Label();
lblProtocol.Text = "协议: ";
lblProtocol.Location = new Point(905, 28);
lblProtocol.AutoSize = true;
groupInput.Controls.Add(lblProtocol);
```

```
cboProtocol = new ComboBox();
cboProtocol.Location = new Point(950, 24);
cboProtocol.Size = new Size(110, 24);
cboProtocol.DropDownStyle = ComboBoxStyle.DropDownList;
cboProtocol.Items.AddRange(new object[] { "TCP", "UDP", "TCP+UDP"
    ↪ });
cboProtocol.SelectedIndex = 0;
groupInput.Controls.Add(cboProtocol);

btnRadius = new Button();
btnRadius.Text = "Radius 1812/1813";
btnRadius.Location = new Point(16, 58);
btnRadius.Size = new Size(135, 26);
btnRadius.Click += BtnRadius_Click;
groupInput.Controls.Add(btnRadius);

btnStart = new Button();
btnStart.Text = "开始扫描";
btnStart.Location = new Point(168, 58);
btnStart.Size = new Size(95, 26);
btnStart.Click += BtnStart_Click;
groupInput.Controls.Add(btnStart);

btnStop = new Button();
btnStop.Text = "停止";
btnStop.Location = new Point(278, 58);
btnStop.Size = new Size(76, 26);
btnStop.Enabled = false;
btnStop.Click += BtnStop_Click;
groupInput.Controls.Add(btnStop);

btnClear = new Button();
btnClear.Text = "清空结果";
btnClear.Location = new Point(369, 58);
btnClear.Size = new Size(90, 26);
btnClear.Click += BtnClear_Click;
groupInput.Controls.Add(btnClear);

btnExport = new Button();
```

```
btnExport.Text = "导出CSV";
btnExport.Location = new Point(474, 58);
btnExport.Size = new Size(90, 26);
btnExport.Click += BtnExport_Click;
groupInput.Controls.Add(btnExport);

progressBar = new ProgressBar();
progressBar.Location = new Point(580, 62);
progressBar.Size = new Size(330, 18);
progressBar.Minimum = 0;
progressBar.Maximum = 100;
groupInput.Controls.Add(progressBar);

lblProgress = new Label();
lblProgress.Text = "就绪";
lblProgress.Location = new Point(920, 62);
lblProgress.Size = new Size(140, 18);
groupInput.Controls.Add(lblProgress);

lvResults = new ListView();
lvResults.Location = new Point(12, 112);
lvResults.Size = new Size(1080, 400);
lvResults.Anchor = AnchorStyles.Top | AnchorStyles.Bottom |
    ↪ AnchorStyles.Left | AnchorStyles.Right;
lvResults.View = View.Details;
lvResults.FullRowSelect = true;
lvResults.GridLines = true;
lvResults.Columns.Add("序号", 55, HorizontalAlignment.Left);
lvResults.Columns.Add("主机", 135, HorizontalAlignment.Left);
lvResults.Columns.Add("端口", 70, HorizontalAlignment.Left);
lvResults.Columns.Add("协议", 70, HorizontalAlignment.Left);
lvResults.Columns.Add("状态", 130, HorizontalAlignment.Left);
lvResults.Columns.Add("说明", 410, HorizontalAlignment.Left);
lvResults.Columns.Add("耗时(ms)", 80, HorizontalAlignment.Left);
lvResults.Columns.Add("线程", 105, HorizontalAlignment.Left);
Controls.Add(lvResults);

Label lblLog = new Label();
lblLog.Text = "运行日志: ";
lblLog.Location = new Point(12, 522);
```

```
        lblLog.AutoSize = true;
        lblLog.Anchor = AnchorStyles.Bottom | AnchorStyles.Left;
        Controls.Add(lblLog);

        txtLog = new TextBox();
        txtLog.Location = new Point(12, 545);
        txtLog.Size = new Size(1080, 125);
        txtLog.Anchor = AnchorStyles.Bottom | AnchorStyles.Left |
            ↪ AnchorStyles.Right;
        txtLog.Multiline = true;
        txtLog.ScrollBars = ScrollBars.Vertical;
        txtLog.ReadOnly = true;
        Controls.Add(txtLog);

        AppendLog("程序启动。");
        AppendLog("PPT要求：图形界面、多线程2-3线程，并测试Radius
            ↪ 1812和1813端口。");
    }

    private void BtnRadius_Click(object sender, EventArgs e)
    {
        nudStartPort.Value = 1812;
        nudEndPort.Value = 1813;
        nudThreadCount.Value = 3;
        cboProtocol.SelectedIndex = 2; // TCP+UDP
        AppendLog("已设置为 Radius 测试端口：1812(Authentication) 和
            ↪ 1813(Accounting)。");
    }

    private void BtnStart_Click(object sender, EventArgs e)
    {
        string host = txtHost.Text.Trim();
        int startPort = (int)nudStartPort.Value;
        int endPort = (int)nudEndPort.Value;

        if (string.IsNullOrEmpty(host))
        {
            MessageBox.Show("请输入目标主机或IP地址。", "提示",
                ↪ MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }
    }
}
```

```
}

if (startPort > endPort)
{
    MessageBox.Show("起始端口不能大于结束端口。", "提示",
        ↪ MessageBoxButtons.OK, MessageBoxIcon.Warning);
    return;
}

int count = endPort - startPort + 1;
if (count > 2048)
{
    DialogResult confirm = MessageBox.Show(
        "端口数量较多，实验报告建议扫描小范围端口。仍然继续吗?",
        "确认扫描范围",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

    if (confirm != DialogResult.Yes)
        return;
}

currentHost = host;
currentTimeoutMs = (int)nudTimeout.Value;
currentMode = GetSelectedMode();

lvResults.Items.Clear();
progressBar.Value = 0;
lblProgress.Text = "0/" + count;
resultIndex = 0;
completedPorts = 0;
totalPorts = count;

portQueue = new ConcurrentQueue<int>();
for (int port = startPort; port <= endPort; port++)
    portQueue.Enqueue(port);

scanCts = new CancellationTokenSource();

SetUiScanning(true);
```



```
AppendLog("开始扫描: " + currentHost + ", 端口范围 " + startPort +
    ↪ "-" + endPort
    + ", 协议 " + cboProtocol.SelectedItem + ", 线程数 " +
    ↪ nudThreadCount.Value + "。");

int workerCount = (int)nudThreadCount.Value;
remainingWorkers = workerCount;
for (int i = 1; i <= workerCount; i++)
{
    string workerName = "ScanWorker-" + i;
    Thread worker = new Thread(() => WorkerLoop(workerName,
        ↪ scanCts.Token));
    worker.IsBackground = true;
    worker.Name = workerName;
    worker.Start();
    AppendLog("启动扫描线程: " + workerName);
}
}

private void BtnStop_Click(object sender, EventArgs e)
{
    if (scanCts != null)
    {
        scanCts.Cancel();
        AppendLog("已发送停止信号, 正在等待正在超时的连接返回。");
    }
}

private void BtnClear_Click(object sender, EventArgs e)
{
    if (btnStart.Enabled)
    {
        lvResults.Items.Clear();
        txtLog.Clear();
        progressBar.Value = 0;
        lblProgress.Text = "就绪";
        AppendLog("已清空结果。");
    }
}
```

```
private void BtnExport_Click(object sender, EventArgs e)
{
    if (lvResults.Items.Count == 0)
    {
        MessageBox.Show("当前没有可导出的扫描结果。", "提示",
            ↪ MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    using (SaveFileDialog dialog = new SaveFileDialog())
    {
        dialog.Filter = "CSV文件|*.csv";
        dialog.FileName = "port_scan_results.csv";
        if (dialog.ShowDialog() == DialogResult.OK)
        {
            StringBuilder csv = new StringBuilder();

            ↪ csv.AppendLine("Index,Host,Port,Protocol,Status,Message,ElapsedMs,V
foreach (ListViewItem item in lvResults.Items)
        {
            string[] fields = new string[item.SubItems.Count];
            for (int i = 0; i < item.SubItems.Count; i++)
                fields[i] = EscapeCsv(item.SubItems[i].Text);
            csv.AppendLine(string.Join(",", fields));
        }

        File.WriteAllText(dialog.FileName, csv.ToString(), new
            ↪ UTF8Encoding(true));
        AppendLog("已导出CSV结果: " + dialog.FileName);
    }
}

private string EscapeCsv(string value)
{
    if (value == null)
        return "";
    bool needQuote = value.Contains(",") || value.Contains("\"") ||
        ↪ value.Contains("\r") || value.Contains("\n");
    value = value.Replace("\"", "\"\"");
```

```
        return needQuote ? "\"" + value + "\"" : value;
    }

    private ScanMode GetSelectedMode()
    {
        if (cboProtocol.SelectedIndex == 1)
            return ScanMode.UDP;
        if (cboProtocol.SelectedIndex == 2)
            return ScanMode.TCPAndUDP;
        return ScanMode.TCP;
    }

    private void WorkerLoop(string workerName, CancellationToken token)
    {
        int port;
        while (!token.IsCancellationRequested && portQueue.TryDequeue(out
            ↪ port))
        {
            if (currentMode == ScanMode.TCP || currentMode ==
                ↪ ScanMode.TCPAndUDP)
            {
                ScanResult tcpResult = ScanTcp(currentHost, port,
                    ↪ currentTimeoutMs, workerName);
                AddResult(tcpResult);
            }

            if (!token.IsCancellationRequested && (currentMode ==
                ↪ ScanMode.UDP || currentMode == ScanMode.TCPAndUDP))
            {
                ScanResult udpResult = ScanUdp(currentHost, port,
                    ↪ currentTimeoutMs, workerName);
                AddResult(udpResult);
            }

            int done = Interlocked.Increment(ref completedPorts);
            UpdateProgress(done);
        }

        int left = Interlocked.Decrement(ref remainingWorkers);
        Ui(() => AppendLog(workerName + " 结束, 剩余工作线程: " + left));
    }
}
```

```
        if (left == 0)
        {
            Ui(() => FinishScan());
        }
    }

private ScanResult ScanTcp(string host, int port, int timeoutMs,
    ↪ string workerName)
{
    Stopwatch stopwatch = Stopwatch.StartNew();
    try
    {
        using (TcpClient client = new TcpClient())
        {
            IAsyncResult ar = client.BeginConnect(host, port, null,
                ↪ null);
            bool connected = ar.AsyncWaitHandle.WaitOne(timeoutMs);
            stopwatch.Stop();

            if (!connected)
            {
                try { client.Close(); } catch { }
                return MakeResult(host, port, "TCP",
                    ↪ "Filtered/Timeout",
                    "TCP connect超时，端口可能被过滤或主机无响应",
                    ↪ stopwatch.ElapsedMilliseconds, workerName);
            }

            try
            {
                client.EndConnect(ar);
                return MakeResult(host, port, "TCP", "Open",
                    "connect连接成功，TCP端口开放",
                    ↪ stopwatch.ElapsedMilliseconds, workerName);
            }
            catch (SocketException ex)
            {
                return MakeResult(host, port, "TCP", "Closed",
```

```
        "connect失败: " + ex.SocketErrorCode,
        ↪ stopwatch.ElapsedMilliseconds, workerName);
    }
}
}
catch (Exception ex)
{
    stopwatch.Stop();
    return MakeResult(host, port, "TCP", "Closed/Error",
        ex.GetType().Name + ": " + ex.Message,
        ↪ stopwatch.ElapsedMilliseconds, workerName);
}
}

private ScanResult ScanUdp(string host, int port, int timeoutMs,
    ↪ string workerName)
{
    Stopwatch stopwatch = Stopwatch.StartNew();

    try
    {
        using (UdpClient udp = new UdpClient())
        {
            udp.Client.ReceiveTimeout = timeoutMs;
            udp.Connect(host, port);

            byte[] probe = Encoding.ASCII.GetBytes("RADIUS-PROBE");
            udp.Send(probe, probe.Length);

            try
            {
                IPEndPoint remote = null;
                byte[] reply = udp.Receive(ref remote);
                stopwatch.Stop();

                string text =
                    ↪ Encoding.ASCII.GetString(reply).Replace("\r", "
                    ↪ ").Replace("\n", " ").Trim();
                if (text.Length > 60)
                    text = text.Substring(0, 60) + "...";
            }
        }
    }
}
```

```
        return MakeResult(host, port, "UDP", "Open",
            "收到UDP响应: " + text,
            ↪ stopwatch.ElapsedMilliseconds, workerName);
    }
    catch (SocketException ex)
    {
        stopwatch.Stop();

        if (ex.SocketErrorCode == SocketError.TimedOut)
        {
            return MakeResult(host, port, "UDP",
                ↪ "Open|Filtered",
                "UDP无响应: 可能开放但未回包, 也可能被防火墙过
                ↪ 滤", stopwatch.ElapsedMilliseconds,
                ↪ workerName);
        }

        if (ex.SocketErrorCode ==
            ↪ SocketError.ConnectionReset)
        {
            return MakeResult(host, port, "UDP", "Closed",
                "收到ICMP端口不可达/连接重置",
                ↪ stopwatch.ElapsedMilliseconds,
                ↪ workerName);
        }

        return MakeResult(host, port, "UDP", "Unknown",
            "UDP异常: " + ex.SocketErrorCode,
            ↪ stopwatch.ElapsedMilliseconds, workerName);
    }
}

catch (Exception ex)
{
    stopwatch.Stop();
    return MakeResult(host, port, "UDP", "Closed/Error",
        ex.GetType().Name + ": " + ex.Message,
        ↪ stopwatch.ElapsedMilliseconds, workerName);
}
```

```
}

private ScanResult MakeResult(string host, int port, string protocol,
    ↪ string status, string message, long elapsedMs, string workerName)
{
    return new ScanResult
    {
        Host = host,
        Port = port,
        Protocol = protocol,
        Status = status,
        Message = message,
        ElapsedMs = elapsedMs,
        Worker = workerName
    };
}

private void AddResult(ScanResult result)
{
    Ui(() =>
    {
        int index = Interlocked.Increment(ref resultIndex);
        ListViewItem item = new ListViewItem(index.ToString());
        item.SubItems.Add(result.Host);
        item.SubItems.Add(result.Port.ToString());
        item.SubItems.Add(result.Protocol);
        item.SubItems.Add(result.Status);
        item.SubItems.Add(result.Message);
        item.SubItems.Add(result.ElapsedMs.ToString());
        item.SubItems.Add(result.Worker);

        if (result.Status.StartsWith("Open"))
            item.ForeColor = Color.DarkGreen;
        else if (result.Status.StartsWith("Closed"))
            item.ForeColor = Color.Firebrick;
        else
            item.ForeColor = Color.DarkOrange;

        lvResults.Items.Add(item);
        lvResults.EnsureVisible(lvResults.Items.Count - 1);
    });
}
```

```
    });  
}  
  
private void UpdateProgress(int done)  
{  
    Ui(() =>  
    {  
        if (totalPorts <= 0)  
            return;  
  
        int percent = (int)Math.Round(done * 100.0 / totalPorts);  
        if (percent < 0) percent = 0;  
        if (percent > 100) percent = 100;  
        progressBar.Value = percent;  
        lblProgress.Text = done + "/" + totalPorts + " (" + percent +  
            ↪ "%)";  
    });  
}  
  
private void FinishScan()  
{  
    SetUiScanning(false);  
    if (scanCts != null && scanCts.IsCancellationRequested)  
        AppendLog("扫描已停止。");  
    else  
        AppendLog("扫描完成。共输出结果行: " + lvResults.Items.Count +  
            ↪ "。");  
  
    scanCts = null;  
}  
  
private void SetUiScanning(bool scanning)  
{  
    btnStart.Enabled = !scanning;  
    btnStop.Enabled = scanning;  
    btnRadius.Enabled = !scanning;  
    btnClear.Enabled = !scanning;  
    btnExport.Enabled = !scanning;  
    txtHost.Enabled = !scanning;  
    nudStartPort.Enabled = !scanning;
```



```
        nudEndPort.Enabled = !scanning;
        nudThreadCount.Enabled = !scanning;
        nudTimeout.Enabled = !scanning;
        cboProtocol.Enabled = !scanning;
    }

    private void AppendLog(string message)
    {
        txtLog.AppendText "[" + DateTime.Now.ToString("HH:mm:ss") + " ] "
            + message + Environment.NewLine);
    }

    private void Ui(Action action)
    {
        if (IsDisposed || !IsHandleCreated)
            return;

        if (InvokeRequired)
            BeginInvoke(action);
        else
            action();
    }

    protected override void OnFormClosing(FormClosingEventArgs e)
    {
        if (scanCts != null)
            scanCts.Cancel();
        base.OnFormClosing(e);
    }
}
```

A.2 PortScannerGUI/Program.cs

```
using System;
using System.Windows.Forms;

namespace PortScannerGUI
{
```

```
internal static class Program
{
    [STAThread]
    private static void Main()
    {
        Application.EnableVisualStyles();
        Application.SetCompatibleTextRenderingDefault(false);
        Application.Run(new MainForm());
    }
}
```

A.3 RadiusMockServer/Program.cs

```
using System;
using System.Net;
using System.Net.Sockets;
using System.Text;
using System.Threading;

namespace RadiusMockServer
{
    internal class Program
    {
        private static volatile bool running = true;

        private static void Main(string[] args)
        {
            Console.OutputEncoding = Encoding.UTF8;
            Console.WriteLine("Radius 1812/1813 测试服务启动。");
            Console.WriteLine("作用：在本机打开 TCP/UDP 1812、1813，便于端口扫描程序截图验证。");
            Console.WriteLine("说明：这是实验辅助程序，不实现真正的 Radius 协议认证。");
            Console.WriteLine("按 Enter 键停止。");

            StartThread(() => TcpListenLoop(1812), "TCP-1812");
            StartThread(() => TcpListenLoop(1813), "TCP-1813");
            StartThread(() => UdpListenLoop(1812), "UDP-1812");
        }
    }
}
```

```
        StartThread(() => UdpListenLoop(1813), "UDP-1813");

        Console.ReadLine();
        running = false;
        Console.WriteLine("正在停止, 请稍候...");
        Thread.Sleep(500);
    }

    private static void StartThread(ThreadStart start, string name)
    {
        Thread thread = new Thread(start);
        thread.IsBackground = true;
        thread.Name = name;
        thread.Start();
        Console.WriteLine("已启动监听线程: " + name);
    }

    private static void TcpListenLoop(int port)
    {
        TcpListener listener = null;
        try
        {
            listener = new TcpListener(IPAddress.Any, port);
            listener.Start();
            Console.WriteLine("TCP " + port + " 正在监听。");

            while (running)
            {
                if (!listener.Pending())
                {
                    Thread.Sleep(100);
                    continue;
                }

                using (TcpClient client = listener.AcceptTcpClient())
                {
                    string remote =
                        ↪ client.Client.RemoteEndPoint.ToString();
                    Console.WriteLine("TCP " + port + " 收到连接: " +
                        ↪ remote);
                }
            }
        }
        catch { }
    }
}
```

```
        byte[] data = Encoding.ASCII.GetBytes("RADIUS-MOCK-OK"
        ↪ " + port + "\r\n");
        client.GetStream().Write(data, 0, data.Length);
    }
}
}
catch (SocketException ex)
{
    Console.WriteLine("TCP " + port + " 监听失败: " +
    ↪ ex.Message);
    Console.WriteLine("请检查端口是否已被占用, 或是否需要关闭防火墙
    ↪ 限制。");
}
finally
{
    if (listener != null)
        listener.Stop();
}
}

private static void UdpListenLoop(int port)
{
    UdpClient udp = null;
    try
    {
        udp = new UdpClient(port);
        Console.WriteLine("UDP " + port + " 正在监听。");

        while (running)
        {
            if (udp.Available == 0)
            {
                Thread.Sleep(100);
                continue;
            }

            IPEndPoint remote = new IPEndPoint(IPAddress.Any, 0);
            byte[] request = udp.Receive(ref remote);
            string text = Encoding.ASCII.GetString(request);
```

```
        Console.WriteLine("UDP " + port + " 收到数据: " + remote
        ↪ + " -> " + text);

        byte[] response = Encoding.ASCII.GetBytes("RADIUS-MOCK-OK
        ↪ " + port);
        udp.Send(response, response.Length, remote);
    }
}
catch (SocketException ex)
{
    Console.WriteLine("UDP " + port + " 监听失败: " +
    ↪ ex.Message);
    Console.WriteLine("请检查端口是否已被占用, 或是否需要关闭防火墙
    ↪ 限制。");
}
finally
{
    if (udp != null)
        udp.Close();
}
}
}
```